一、(26%) How pipelining affects the clock cycle time of the processor. Assume that individual stages of the datapath have the following latencies:

IF	ID	EX	MEM	WB
450ps	350ps	550ps	400ps	500ps

Also, assume that instructions executed by the processor are broken down as follows:

ALU/Logic	Jump/Branch	Load	Store
45%	20%	20%	15%

1.1. (6%) What is the clock cycle time in a pipelined and non-pipelined processor?

Pipeline : max (450, 350, 550, 400, 500) = 550 ps #

Non-Pipeline: 450 + 350 + 550 + 400 + 500 = 2250 ps #

1.2. (7%) What is the total latency of an LW instruction in a pipelined and non-pipelined processor?

Pipeline : 550 × 5 = 2750 ps #

Non-Pipeline: 450 + 350 + 550 + 400 + 500 = 2250 ps #

1.3. (7%) If we can split one stage of the pipelined datapath into two new stages, each with half the latency of the original stage, which stage would you split and what is the new clock cycle time of the processor?

Split the largest one : 550 to two 225 cycles

New clock rate = max (450, 350, 225, 225, 400, 500)

= 500ps ANS: Split EX stage, 500ps #

1.4. (3%) Assuming there are no stalls or hazards, what is the utilization of the data memory?

20% (lw) + 15% (sw) = 35 % #

1.5. (3%) Assuming there are no stalls or hazards, what is the utilization of the write-register port of the "Registers" unit?

45% (ALU (R-Type)) + 20% (lw) = 65 % #

二、(6%) This exercise examines the accuracy of various branch predictors for the following repeating pattern (e.g., in a loop) of branch outcomes: T, T, NT, NT, T, T, NT. What is the accuracy of always-taken and always-not-taken predictors for this sequence of branch outcomes?

Always - taken : $\frac{4}{7} \cong 57.14\%$ #

Always - not - taken : $\frac{3}{7} \cong 42.86\%$ #

三、(20%) Answer the following questions:

3.1. (5%) If there is forwarding, is NOP unnecessary? Why or why not?

ANS: No，因為 forwarding 無法消除 load-use hazard

3.2 (5%) There are three primary types of data dependencies in a program: **RAW**, **WAR**, and **WAW**.Which of these three dependencies can cause a data hazard in a conventional MIPS-like instruction pipeline? (Provide one example in MIPS assembly code that illustrates the identified data hazard. Clearly mark the line where the hazard occurs and state the type of dependency.)

ANS: RAW 會導致 data hazard，以下為範例 assembly

```
add ($2), $3, $4
sub $1, ($2), $5   → 此時助 $2 尚未寫於 $3+$4，因為其尚未被 WB
```

3.3. (5%) What is the minimum number of cycles needed to completely execute n instructions on a CPU with a k stage pipeline? Justify your formula.

ANS: (1) $n+k-1$

(2) n 條指令助開頭會佔用 n 個 CLK，而最後一條指令會需要 k-1 個 CLK 來完成

3.4 (5%) What is the minimum number of clock cycles needed to completely execute n instructions with a k stage non-pipelined processor? Justify your formula.

ANS: (1) $n \times k$

(2) 每條指令皆需等待前指令完成，且每條指令需要 k

四、(20%) This exercise is intended to help you understand the relationship between forwarding, hazard detection, and ISA design. Problems in this exercise refer to the following sequence of instructions, and assume that it is executed on a 5-stage pipelined datapath:

I1: lw r5,4(r5)
I2: add r5,r2,r5
I3: lw r3,0(r5)
I4: or r3.r5.r3
I5: sw r4,0(r5)

4.1. (5%) Identify if there are any **RAW, WAR** or **WAW** dependencies? Please list the two instructions involved, e.g., (I6, I7).

ANS: (1) RAW : (I1, I2) 、(I2, I3) 、(I3, I4)

(2) WAR : (I1, I2)

(3) WAW : (I1, I2) 、(I3, I4)

4.2.(5%) If there is no forwarding or hazard detection, how many **nops** needed to be inserted to ensure correct execution, and how many cycles needed to execute the codes?

ANS: 6 nop command and 15 cycles needed※

4.3.(5%) If the processor has full forwarding and hazard detection, how many **nops** still needed to be inserted to ensure correct execution, and how many cycles needed to execute the codes?

ANS : 2nops and 11 cycles

4.4. (5%) If the processor **has forwarding**, but we **forgot to implement the load-use hazard detection unit**, what happens when this code executes?

ANS: load 指令無法在暫存器被存取前寫入正確的值，導致下條的 use 指令存取到非預期的值。

五、 (25%) This exercise is intended to help you understand the relationship between delay slots, control hazards, and branch execution in a pipelined processor. In this exercise, we assume that the following MIPS code is executed on a pipelined processor with a 5-stage pipeline, full forwarding, and a predict-taken branch predictor:

```
1          lw r2,0(r1)
2 label1:  beq r2,r0,label2 # not taken once, then taken
3          lw r3,0(r2)
4          beq r3,r0,label1 # taken
5          add r1,r3,r1
6 label2:  sw r1,0(r2)
```

5.1 (10%) Assuming there are no delay slots and that branches **decide in the MEM stage**. How many cycles will it take to complete the above MIPS codes?

① lw : IF ID EX MEM WB

② beq : IF ID IDn EX MEM WB

⑥ sw : IF ID EX flushed

⑦ sw+4 : IF ID flushed

⑧ sw+8 : IF flushed

③ lw : IF ID EX MEM WB

④ beq : IF ID nop ALU DM REG

② beq : IM REG ALU DM REG

⑥ sw : IM REG ALU DM REG

ANS: 15

5.2. (10%) How many cycles will it take to complete? Assume that delay slots are used. In the given code, the instruction following the branch is now the delay slot instruction for that branch.
(Hint: Branches **decide** in the ID stage)

① lw : IF ID EX MEM WB

② beq : IF ID IDn IDn EX MEM WB

③ lw : IF IFn IFn ID EX MEM WB

④ beq : IF ID IDn IDn EX MEM WB

⑤ add : IF IFn IFn ID EX MEM WB

② beq : IF ID EX MEM WB

③ lw : IF ID EX MEM WB

⑥ sw : IF ID EX MEM WB

ANS: 16

5.3 (5%) In part 5.2, can the compiler always find a useful instruction to fill the delay slot? What happens if no useful instruction can be found?

ANS: 不一定，編譯器會填入 nop 指令

六、(10%) Consider the following code segment in C:

```
a = b + e;
c = b + f;
```

Here is the generated MIPS code for this segment, assuming all variables are in memory and are addressable as offsets from $t7:

```
lw $t0, 0($t7) # Load b
lw $t1, 8($t7) # Load e
add $t2, $t0, $t1 # b + e
sw $t2, 24($t7) # Store a
lw $t3, 16($t7) # Load f
add $t4, $t0, $t3 # b + f
sw $t4, 32($t7) # Store c
```

Find the hazards in the preceding code segment and reorder the instructions to avoid any pipeline stalls.

ANS: (1)

```
lw $t0, 0($t7) # Load b
lw $t1, 8($t7) # Load e
add $t2, $t0, $t1 # b + e      <- load-use hazard
sw $t2, 24($t7) # Store a
lw $t3, 16($t7) # Load f
add $t4, $t0, $t3 # b + f      <- load-use hazard
sw $t4, 32($t7) # Store c
```

(2)

```
lw $t0, 0($t7)
lw $t1, 8($t7)
lw $t3, 16($t7)
add $t2, $t0, $t1
sw $t2, 24($t7)
add $t4, $t0, $t3
sw $t4, 32($t7)
```